

CMPE 125 — DIGITAL DESIGN

Lab 3 Report

Hexadecimal Seven-Segment Decoder

Name: Yixu Liu **Student ID:** 017406857
Lab: 3 **Date:** September 15, 2026

1 Logic Circuit Design and Calculations

The circuit converts a four-bit input, $SW[3:0]$, into the seven segment signals used to display hexadecimal inputs 0 through F. The input variables are D_3, D_2, D_1, D_0 , with D_3 as the most significant bit. The output order is

$$HEX0[6:0] = \{s_g, s_f, s_e, s_d, s_c, s_b, s_a\}.$$

The segment outputs are active low: a logic 0 turns a segment on, and a logic 1 turns it off. The digit-enable pattern is $AN = 4'b1110$. Each segment is implemented as a combinational Boolean function of the four input bits.

1.1 Design Method

The truth table lists the required segment pattern for each of the 16 input combinations. For each output, the cells containing 1 are grouped in a four-variable Karnaugh map. The row order for D_3D_2 and the column order for D_1D_0 are both 00, 01, 11, 10. Adjacent cells are combined to eliminate changing variables, producing the sum-of-products expressions below. The original truth table and handwritten calculations are included on the following pages.

1.2 Boolean Expressions

In these equations, a bar represents NOT, multiplication represents AND, and + represents OR. The expressions correspond to the supplied design.

$$\begin{aligned} s_a &= D_2\overline{D_1}D_0 + D_3D_2\overline{D_1} + \overline{D_3}D_2\overline{D_1}D_0 + D_3\overline{D_2}D_1D_0, \\ s_b &= D_3D_2\overline{D_0} + \overline{D_3}D_2\overline{D_1}D_0 + D_3D_1D_0 + D_2D_1\overline{D_0}, \\ s_c &= \overline{D_3}D_2D_1\overline{D_0} + D_3D_2\overline{D_0} + D_3D_2D_1, \\ s_d &= \overline{D_3}D_2\overline{D_1}D_0 + \overline{D_2}D_1D_0 + D_2D_1D_0 + D_3\overline{D_2}D_1\overline{D_0}, \\ s_e &= \overline{D_3}D_0 + \overline{D_3}D_2\overline{D_1} + \overline{D_2}D_1D_0, \\ s_f &= D_3D_2\overline{D_1} + \overline{D_3}D_2D_0 + \overline{D_3}D_2D_1 + \overline{D_3}D_1D_0, \\ s_g &= \overline{D_3}D_2\overline{D_1} + \overline{D_3}D_2D_1D_0. \end{aligned}$$

These equations are implemented with continuous assign statements. The corresponding logic consists of inverters, AND gates for each product term, and an OR gate for each segment output.

1.3 Original Truth Table

The image shows a handwritten truth table on a digital notepad. The table has 13 columns: 'Hex digit', 'input' (D3, D2, D1, D0), 'Output s' (sg, sf, se, sd, sc, sb, sa), and 'in hex'. The rows correspond to hexadecimal digits 0 through F. The output bits are arranged in a specific order that corresponds to the hexadecimal value. For example, for hex digit 0, the output bits are sg=1, sf=0, se=0, sd=0, sc=0, sb=0, sa=0, which equals 40 in hex. For hex digit F, the output bits are sg=0, sf=0, se=0, sd=1, sc=1, sb=1, sa=0, which equals 15 in hex.

Hex digit	input				Output s							in hex
	D ₃	D ₂	D ₁	D ₀	s _g	s _f	s _e	s _d	s _c	s _b	s _a	
0	0	0	0	0	1	0	0	0	0	0	0	40
1	0	0	0	1	1	1	1	1	0	0	1	29
2	0	0	1	0	0	1	0	0	1	0	0	24
3	0	0	1	1	0	1	1	0	0	0	0	30
4	0	1	0	0	0	0	1	1	0	0	1	19
5	0	1	0	1	0	0	1	0	0	1	0	12
6	0	1	1	0	0	0	0	0	0	1	0	2
7	0	1	1	1	1	1	1	1	0	0	0	78
8	1	0	0	0	0	0	0	0	0	0	0	0
9	1	0	0	1	0	0	1	1	0	0	0	18
A	1	0	1	0	0	0	0	1	0	0	0	8
B	1	0	1	1	0	0	0	0	0	1	1	3
C	1	1	0	0	0	1	0	0	1	1	1	27
D	1	1	0	1	0	1	0	0	0	0	1	21
E	1	1	1	0	0	0	0	0	1	1	0	6
F	1	1	1	1	0	0	0	1	1	1	0	15

Figure 1: Handwritten truth table for hexadecimal inputs 0–F, with outputs in $s_g s_f s_e s_d s_c s_b s_a$ order and the equivalent hexadecimal codes.

1.4 Karnaugh Maps for Segments a, b, and c

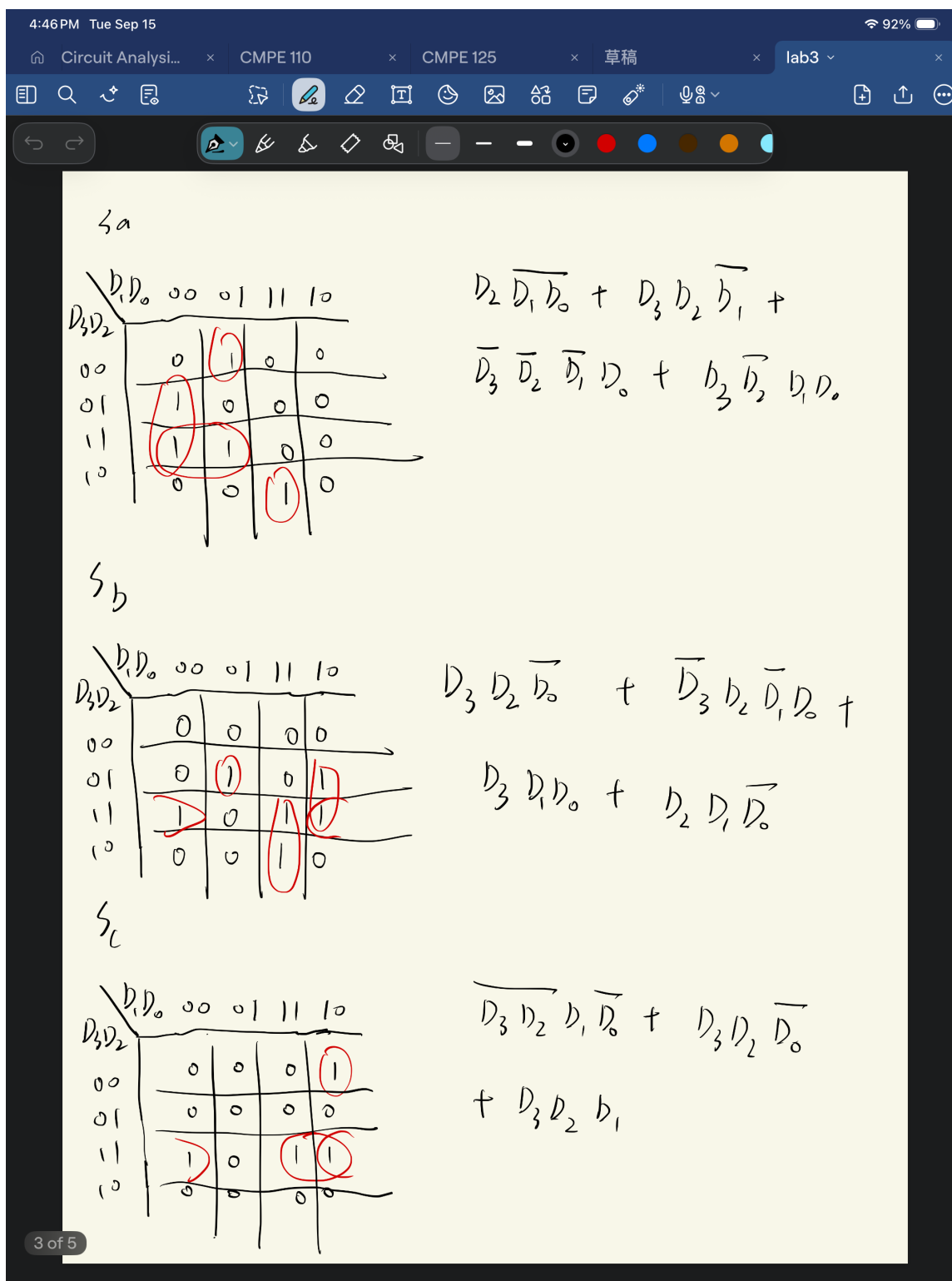


Figure 2: Original Karnaugh-map groupings and Boolean calculations for segments a, b, and c.

1.5 Karnaugh Maps for Segments d, e, and f

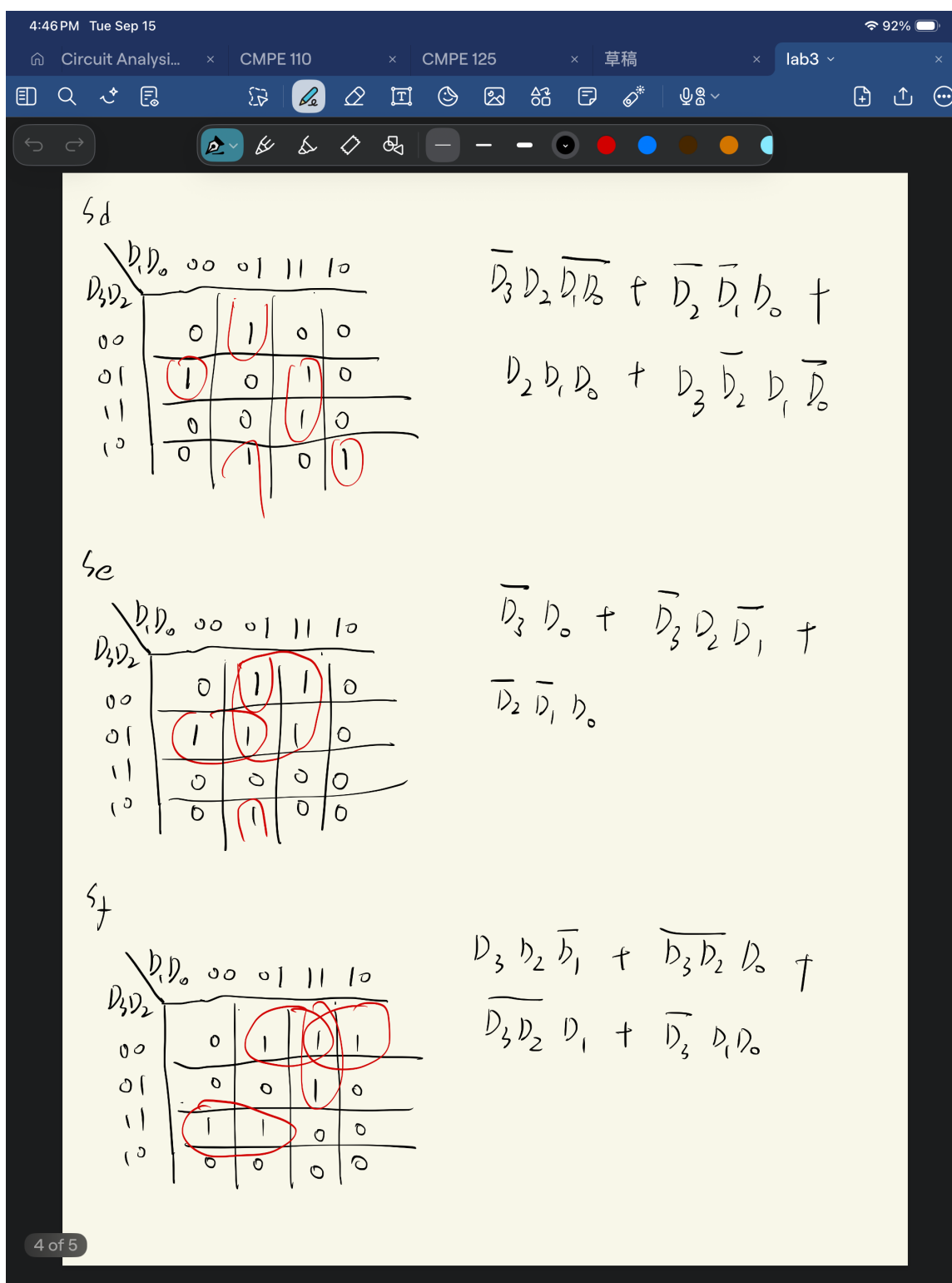


Figure 3: Original Karnaugh-map groupings and Boolean calculations for segments d, e, and f.

1.6 Karnaugh Map for Segment g

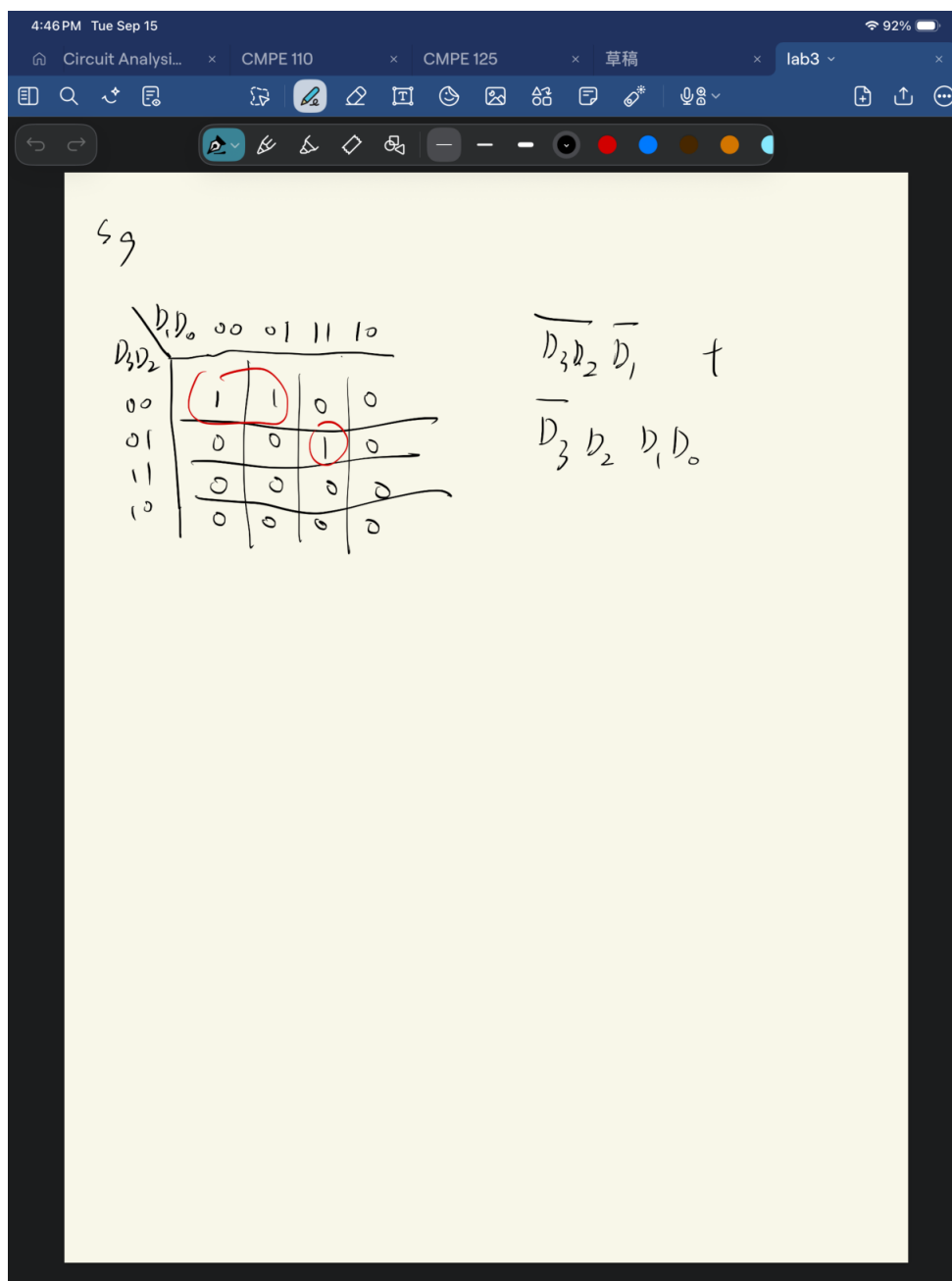


Figure 4: Original calculation for segment g. The source image is named `kmap_e.PNG`, but the handwritten work is labeled s_g .

For segment g, the output is 1 for input values 0, 1, and 7. Grouping the cells for 0 and 1 gives $\overline{D_3D_2D_1}$; the isolated cell for 7 gives $\overline{D_3D_2D_1D_0}$.

2 Output Simulation Waveforms

The supplied testbench applies successive input values at 100 ns intervals. Figure 5 shows the input bus and the segment outputs.

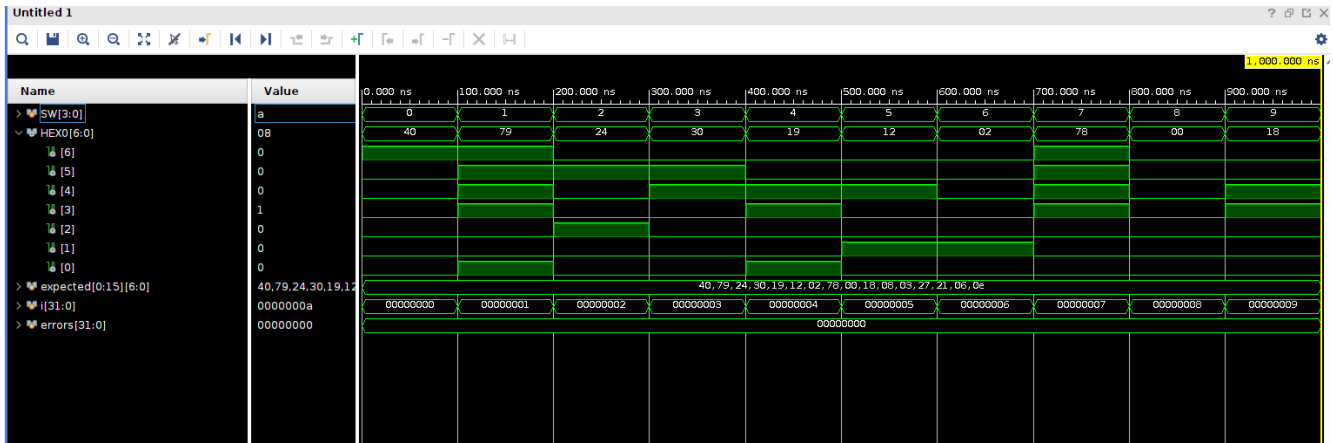


Figure 5: Supplied simulation waveform screenshot. The visible intervals show inputs 0–9 and their corresponding output codes.

3 Verification Summary

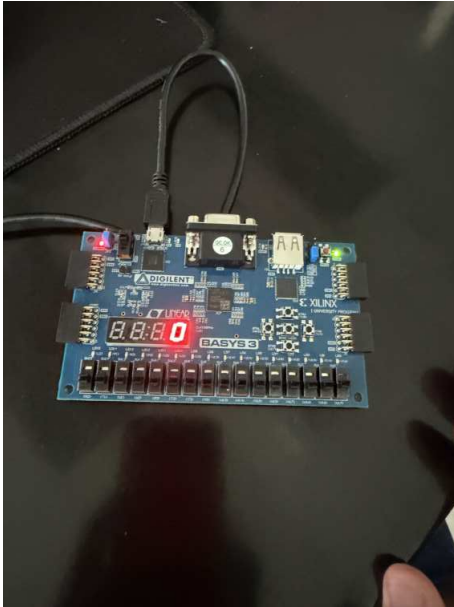
The implemented logic was verified by comparing the simulated segment outputs with the truth table. In the displayed waveform, SW[3:0] advances every 100 ns, and HEX0[6:0] follows the hexadecimal output sequence 40, 79, 24, 30, 19, 12, 02, 78, 00, 18 for inputs 0 through 9. For example, input 1 produces 7'b1111001, which turns on segments b and c, while input 8 produces 7'b0000000, which turns on all seven segments. The displayed simulation results agree with the active-low patterns in the design truth table and support the correct operation of the decoder for the cases shown. The hardware photographs in the appendix document the displayed characters for hexadecimal inputs 0 through F.

4 Separate Verilog Source Files

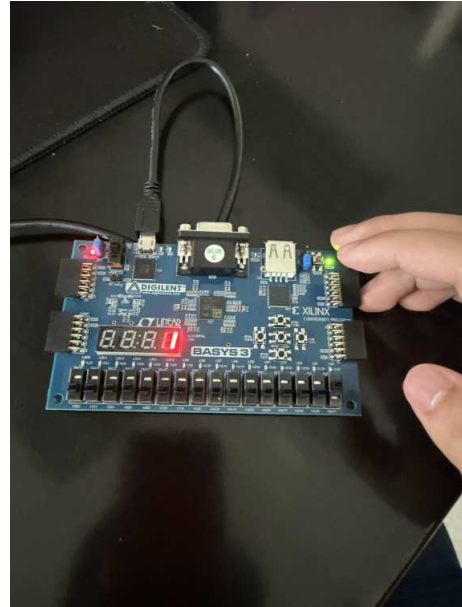
The original design file, seven_segment.v, is provided separately with this report. The original testbench, seven_segment_tb.v, is also included. Both files are supplied unchanged.

A Hardware Output Photographs

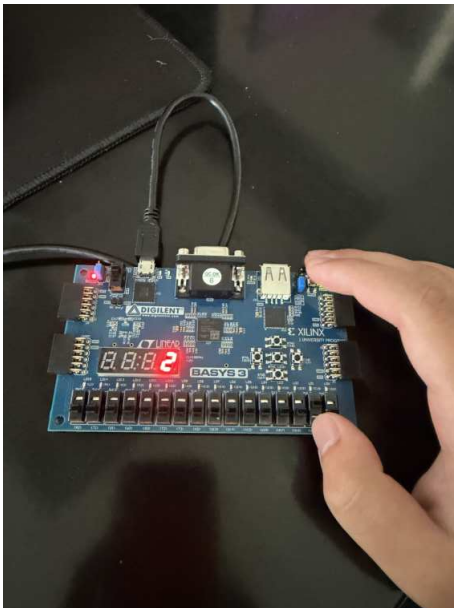
The supplied photographs show the seven-segment display on the Basys 3 board for each hexadecimal input.



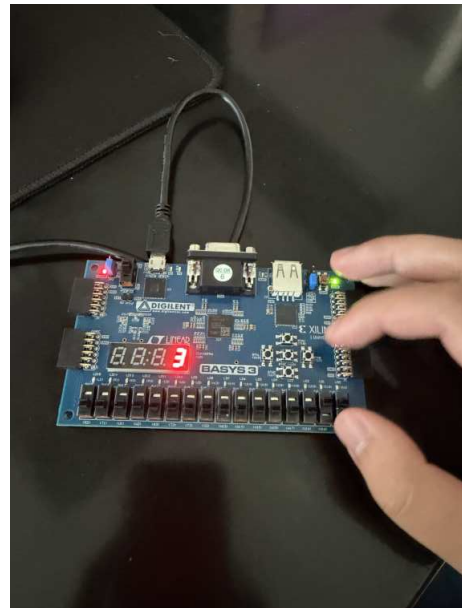
Input 0



Input 1



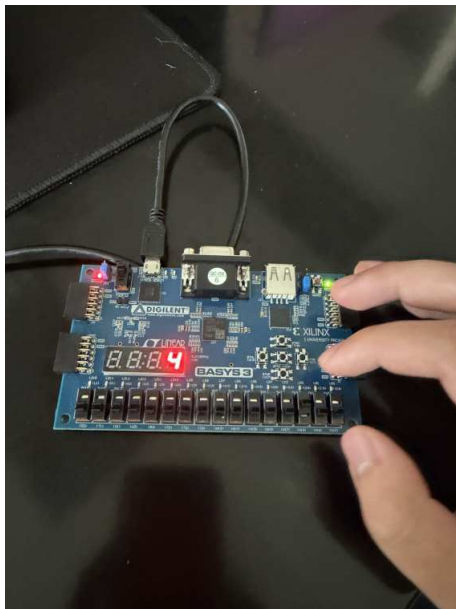
Input 2



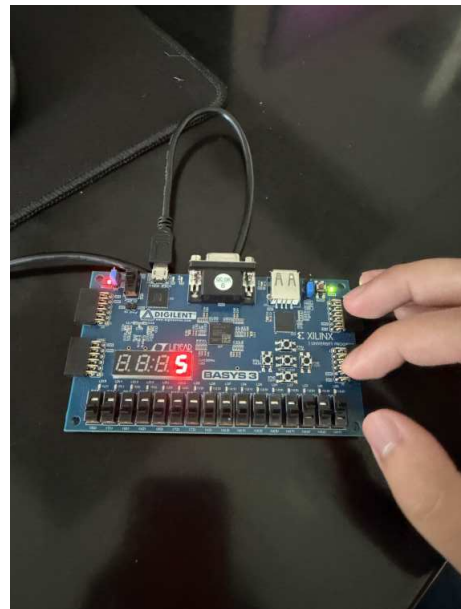
Input 3

Figure 6: Hardware display photographs for inputs 0–3.

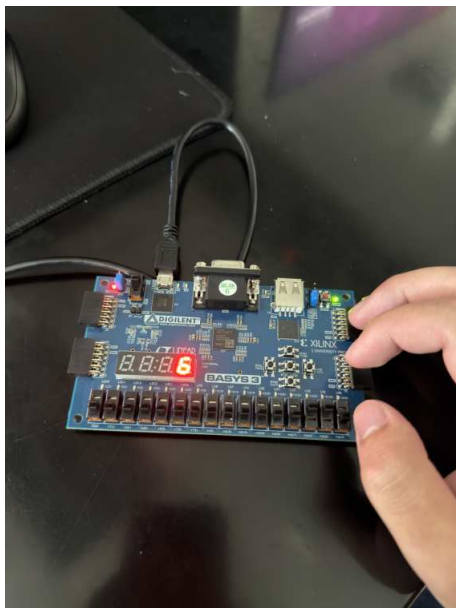
Hardware Output Photographs (continued)



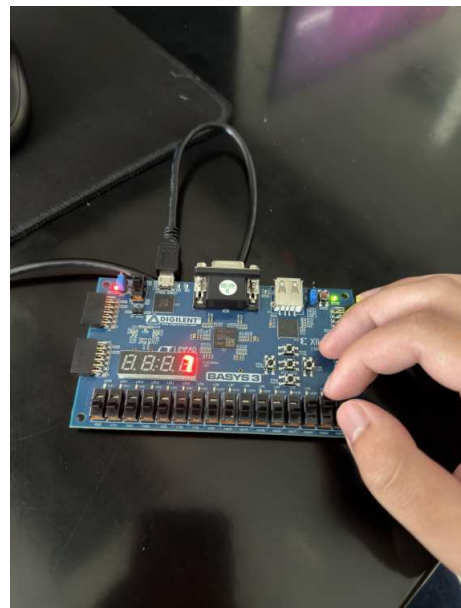
Input 4



Input 5



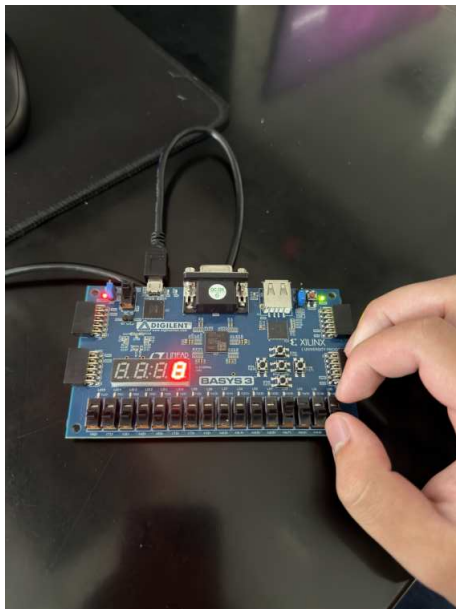
Input 6



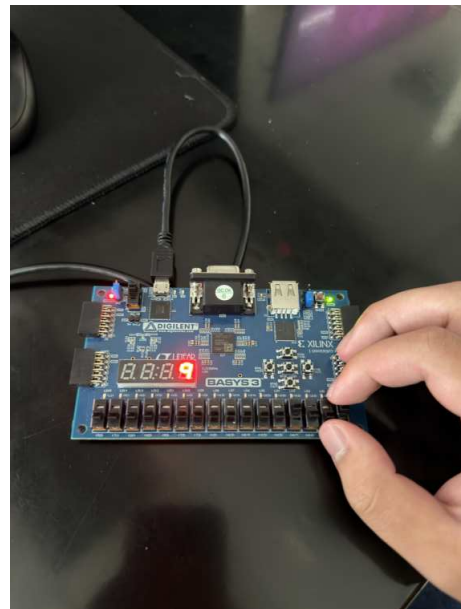
Input 7

Figure 7: Hardware display photographs for inputs 4–7.

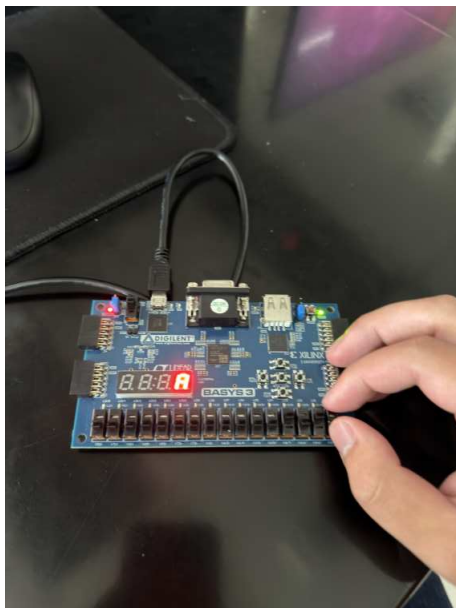
Hardware Output Photographs (continued)



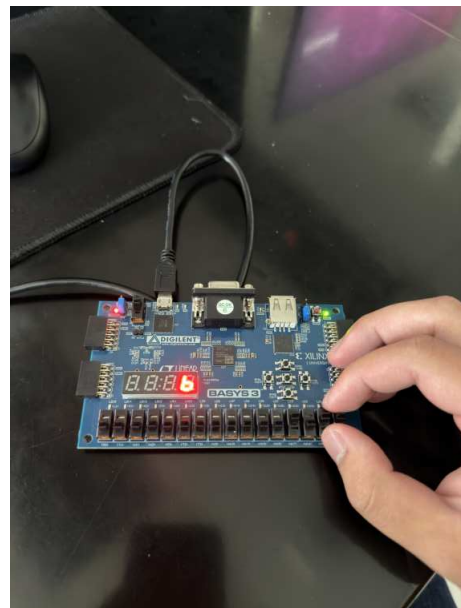
Input 8



Input 9



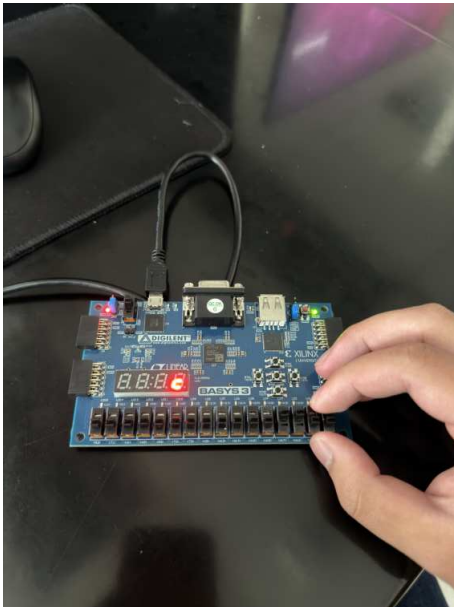
Input A



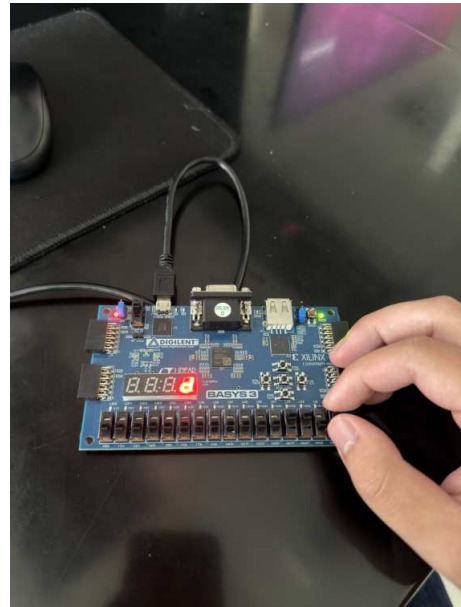
Input B

Figure 8: Hardware display photographs for inputs 8, 9, A, and B.

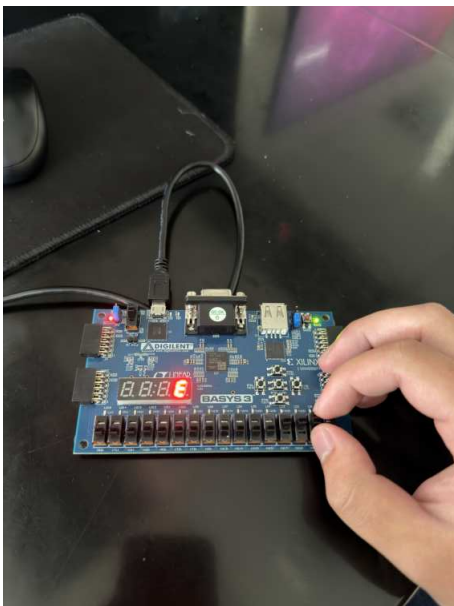
Hardware Output Photographs (continued)



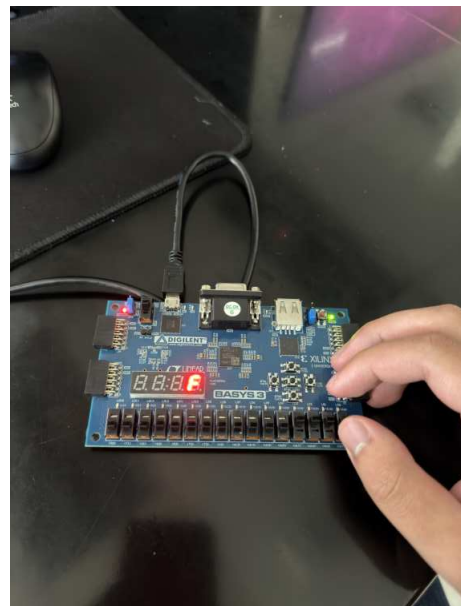
Input C



Input D



Input E



Input F

Figure 9: Hardware display photographs for inputs C–F.